G7_Task

Hello,

Please read the following carefully:

As a Software Engineering research team at the Polytechnique Montréal (Canada), we are interested in understanding developers' assessment of coding task difficulty. You will be provided with a series of 12 coding questions. Questions are divided into two parts:

- First, you will be given three instructions representing coding tasks and you will be asked to write, in a few words, what is the underlying task represented by all these instructions. The instructions represent the same tasks, but they vary in wording and information they give. In a nutshell, you have to distill the essence of the task and write it in a few words.

- In a second time, you will be asked to assess how difficult implementing this ***task*** would be. Note we ask the assessment based on the task you will have worded in the previous part, not based on any of the instructions presented earlier. First, you will be asked your subjective judgment on a scale from Very Easy to Very Difficult. Then, to make judging the difficulty more relatable, we adopt a similar approach as what is used in agile development and planning poker, i.e. you will be asked to evaluate how long it would take to implement a task (in minutes here). Consider in your assessment the knowledge needed to implement the task, the external information you would need to use, whether you would need the assistance of a LLM such as ChatGPT (and so the time it would take to debug the obtained code) etc. Finally, you will have to explain in a few words why you gave such difficulty ratings.

To give you an idea of the range of tasks you will need to assess, the first section of this questionnaire will be two examples we devised to illustrate the questionnaire.

Completing the task will take about 30 minutes.

Note also that all examples are written using Python 3.10 syntax.

Please make sure to answer all the questions to the best of your ability.

Please note that your identity and personal data will not be disclosed, while we plan to use the aggregated results and anonymized responses as part of a scientific publication. If you have any questions about the questionnaire or our research, please do not hesitate to contact us.

By proceeding, you consent to take part in the study under the conditions described previously.

Thank you,

Florian Tambon (florian-2.tambon@polymtl.ca)
Cyrine Zid (cyrine.zid@polymtl.ca)
Amin Nikanjam (amin.nikanjam@polymtl.ca)
Foutse Khomh (foutse.khomh@polymtl.ca)
Giuliano Antoniol (giuliano.antoniol@polymtl.ca)

---

* Indique une question obligatoire

1. **Please enter your prolific ID :** *

Example of tasks

As we ask of you some estimation of difficulty and the tasks dealt in this questionnaire can be a bit different from what you can be used to as a developer, we give you examples of some tasks along with some assessments.

Note that this assessment is **our assessment**. You may have a different opinion based on your background. The idea of those examples is to give an extreme range of what type of tasks you can expect in the questionnaire.

As a reminder: You will first be given three instructions and asked to explain what is the task is about in a few words. Then, you will have to rate the difficulty of the ***task*** you just described (not accounting for particular instruction). Rating the difficulty is done in two steps: subjective difficulty and approximate time needed. Finally, you will be asked to say why you gave such rating.

Example 1

Instruction 1

```
def subtract(c1, c2):
        """
        Deliver the result of subtracting two complex numbers,
        specified as 'c1' and 'c2', as a complex output.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        """
```

Instruction 2

```
def subtract(c1, c2):
        """
        The method receives two complex numbers 'c1' and 'c2', from which it
        subtracts 'c2.real' from 'c1.real' to determine the real portion, and
        subtracts 'c2.imag' from 'c1.imag' to determine the imaginary portion.
        It constructs and returns a new complex number using these results
        with the 'complex()' function.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        """
```

Instruction 3

```
def subtract(c1, c2):
        """
        Subtracts two complex numbers.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        >>> complexCalculator = ComplexCalculator()
        >>> complexCalculator.subtract(1+2j, 3+4j)
        (-2-2j)
        """
```

Answer

The task encapsulated by the **subtract**

function

is to compute the subtraction between two complex numbers.

This task has a subjective difficulty of **Very Easy** as assessed by us and would take about **1 min**
to implement. There is no particular difficulty, the task is very straightforward without any corner case, it just requires to know what a complex is, and the standard Python library for it.

Example 2

Context

```python
import numpy as np
class MetricsCalculator2:
    """
    The class provides to calculate Mean Reciprocal Rank (MRR) and Mean Average Precision (MAP)
    based on input data, where MRR measures the ranking quality and MAP measures the average precision.
    """

    def __init__(self):
        pass

    @staticmethod
    def map(data):
        pass
```

Instruction 1

```python
def mrr(data):
        """
        Compute the MRR of the input data. MRR is a widely used evaluation index.
        It is the mean of reciprocal rank.
        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.
        >>> MetricsCalculator2.mrr(([1, 0, 1, 0], 4))
        >>> MetricsCalculator2.mrr(([([1, 0, 1, 0], 4), ([0, 1, 0, 1], 4)])
        1.0, [1.0]
        0.75, [1.0, 0.5]

        """
```

Instruction 2

```python
def mrr(data):
        """
        Assess the Mean Reciprocal Rank (MRR) from the input 'data'.
        MRR assesses the average of reciprocal ranks of outcomes.
        The input structure 'data' must be a tuple consisting of a list of
        binary values and its total count or a list of such tuples.
        Each binary value (0 or 1) represents the correctness of an answer.
        If 'data' is a tuple, the function returns its mean reciprocal rank.
        If 'data' is a list, the function returns the overall average MRR along
        with a list showing reciprocal ranks for each component tuple.

        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.

        """
```

Instruction 3

```
def mrr(data):
        """
        Evaluate the Mean Reciprocal Rank (MRR) from 'data'. This metric calculates the mean of reciprocal
        ranks for correct answers. Input 'data' could be either a tuple containing binary correctness indicators
        and the aggregate count of answers, or multiple tuples of this type in a list. For each tuple, the function
        measures reciprocal ranks by multiplying the binary indicators by reciprocals of their positions, finding the
        earliest nonzero value to establish the rank of the first correct response. If the input 'data' is a tuple,
        the MRR calculated for this specific tuple is returned; if it's a list, the function summarizes by providing
        both the average MRR and a list of individual MRR values for each tuple.

        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.

        """
```

Answer

The task encapsulated by the **mrr** function is to compute the
Mean Reciprocal Rank of binary tuples data.

This task has a subjective difficulty of **Very Hard** as assessed by us and would take about **40 min**
to implement. The difficulty mainly branches from dealing with the
tuple data and how the formula is processed accordingly (with a few corner cases). This can take
additional time if the Mean Reciprocal Rank is not known.

Question 1:

Below are instructions for implementing the function.

Instruction 1

```
def maximum(arr, k):
    """
    Write a function named "maximum" which takes two parameters:
    a list of integers "arr" and a positive integer "k".
    The function aims to return a list of the "k" largest
    integers from "arr" sorted in ascending order.
    Initially, the function sorts "arr" in descending order using
    "sorted(arr)[::-1]". From this sorted list, it then extracts
    the first "k" elements using slicing "[:k]". Finally, it returns
    these "k" elements sorted in ascending order by applying
    "sorted()" to the sliced list. This ensures the top "k"
    elements are both extracted and returned in the correct order.
   """
```

Instruction 2

```
def maximum(arr, k):
    """
    Given an array arr of integers and a positive integer k,
    return a sorted list of length k with the maximum k
    numbers in arr.

    Example 1:

        Input: arr = [-3, -4, 5], k = 3
        Output: [-4, -3, 5]

    Example 2:

        Input: arr = [4, -4, 4], k = 2
        Output: [4, 4]

    Example 3:

        Input: arr = [-3, 2, 1, 2, -1, -2, 1], k = 1
        Output: [2]

    Note:
        1. The length of the array will be in the range of [1, 1000].
        2. The elements in the array will be in the range of [-1000, 1000].
        3. 0 <= k <= len(arr)
    """
```

Instruction 3

```
def maximum(arr, k):
    """
    Develop a function named 'maximum' which returns a
    sorted list comprising the top 'k' largest integers
    from a specified list of integers 'arr'. The list
    returned should have a length of 'k'.
    """
```

2. In a few words, what is the task this function aims to implement? *

3. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

4. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ◯ 1 min
- ◯ 5 min
- ◯ 10 min
- ◯ 20 min
- ◯ 40 min
- ◯ > 40 min

5. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 2:

Below are instructions for implementing the function:

Context

```python
import xml.etree.ElementTree as ET
class XMLProcessor:
    """
    This is a class as XML files handler, including reading, writing,
    processing as well as finding elements in a XML file.
    """

    def __init__(self, file_name):
        """
        Initialize the XMLProcessor object with the given file name.
        :param file_name:string, the name of the XML file to be processed.
        """
        self.file_name = file_name
        self.root = None

    def read_xml(self):
        pass

    def write_xml(self, file_name):
        pass

    def find_element(self, element_name):
        pass
```

Instruction 1

```
def process_xml_data(self, file_name):
  """
  Update XML elements by converting their text to uppercase, and save
  the updated XML to a new file specified by the 'file_name'.
  The function returns 'True' if it successfully writes the file,
  otherwise it returns 'False'. The 'process_xml_data' function
  iterates through 'self.root.iter('item')' to access each item's
  text ('element.text'), transforms this text to uppercase
  ('text.upper()'), and resets it to 'element.text'.
  It then attempts to write the altered XML data to the specified
  file using 'self.write_xml(file_name)'.
  :param file_name: string, the name of the file to write the
                                                    modified XML data.
  :return: bool, True if the write operation is successful, False otherwise.
  """
```

Instruction 2

```
def process_xml_data(self, file_name):
  """
  Modifies the data in XML elements and writes the updated XML data to a new file.
  :param file_name: string, the name of the file to write the modified XML data.
  :return: bool, True if the write operation is successful, False otherwise.
  >>> xml_processor = XMLProcessor('test.xml')
  >>> root = xml_processor.read_xml()
  >>> success = xml_processor.process_xml_data('processed.xml')
  >>> print(success)
  True
  """
```

Instruction 3

```
def process_xml_data(self, file_name):
  """
  Process XML by changing the text of all 'item' elements in the root to
  uppercase and subsequently save the changes to a file given by 'file_name'.
  The function is designed to return 'True' on a successful write operation
  and 'False' otherwise. It loops through 'item' elements in the root, converts
  the text and writes the updated XML to the file denoted by 'file_name'.
  :param file_name: string, the name of the file to write the modified XML data.
  :return: bool, True if the write operation is successful, False otherwise.
  """
```

6. In a few words, what is the task this function aims to implement? *

7. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ( ) Very Easy
- ( ) Easy
- ( ) Not Easy Nor Difficult
- ( ) Difficult
- ( ) Very Difficult

8. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ( ) 1 min
- ( ) 5 min
- ( ) 10 min
- ( ) 20 min
- ( ) 40 min
- ( ) > 40 min

9. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 3:

Below are instructions for implementing the function:

Instruction 1

```
from typing import List, Tuple

def sum_product(numbers: List[int]) -> Tuple[int, int]:
    """
    Construct a function called 'sum_product' accepting a list of integers for processing.
    This function aims to deliver two results from the input list: the aggregate sum and
    the overall product of the integers. It begins with two initialized variables:
    the sum at zero and the product at one. As it cycles through each number in the list,
    it adds to the sum and multiplies into the product accordingly. Should there be no
    integers in the list, the sum and product are kept at zero and one respectively.
    The function concludes by returning a tuple of these values, sum and product
    in that order, adeptly handling both instances of empty and non-empty lists.
    """
```

Instruction 2

```
from typing import List, Tuple

def sum_product(numbers: List[int]) -> Tuple[int, int]:
    """
    For a given list of integers, return a tuple consisting
    of a sum and a product of all the integers in a list.
    Empty sum should be equal to 0 and empty product should be
    equal to 1.
    >>> sum_product([])
    (0, 1)
    >>> sum_product([1, 2, 3, 4])
    (10, 24)
    """
```

Instruction 3

```
from typing import List, Tuple

def sum_product(numbers: List[int]) -> Tuple[int, int]:
    """
    Develop a function titled 'sum_product', which processes a list of integers named 'numbers'.
    Its objective is to determine the sum and product of the elements in this list, beginning
    with initial values of 0 for sum ('s') and 1 for product ('p'). By iterating over each integer
    in 'numbers', it sequentially adds to 's' and multiplies 'p'. Post loop completion, the function
    issues a tuple with 's' and 'p'. It effectively manages scenarios with an empty input list by
    keeping the output consistent at (0, 1).
    """
```

10. In a few words, what is the task this function aims to implement? *

_____

11. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- [ ] Very Easy
- [ ] Easy
- [ ] Not Easy Nor Difficult
- [ ] Difficult
- [ ] Very Difficult

12. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- [ ] 1 min
- [ ] 5 min
- [ ] 10 min
- [ ] 20 min
- [ ] 40 min
- [ ] > 40 min

13. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 4:

Below are instructions for implementing the function:

Context

```python
import xml.etree.ElementTree as ET
class XMLProcessor:
    """
    This is a class as XML files handler, including reading, writing,
    processing as well as finding elements in a XML file.
    """

    def __init__(self, file_name):
        """
        Initialize the XMLProcessor object with the given file name.
        :param file_name:string, the name of the XML file to be processed.
        """
        self.file_name = file_name
        self.root = None

    def read_xml(self):
        pass

    def process_xml_data(self, file_name):
        pass

    def find_element(self, element_name):
        pass
```

Instruction 1

```python
def write_xml(self, file_name):
        """
        Design a routine that writes the XML data into a specified file called
        'file_name'. If successful, it should return 'True'; if it fails,
        it should return 'False'.
        :param file_name: string, the name of the file to write the XML data.
        :return: bool, True if the write operation is successful, False otherwise.
        """
```

Instruction 2

```
def write_xml(self, file_name):
        """
        Writes the XML data to the specified file.
        :param file_name: string, the name of the file to write the XML data.
        :return: bool, True if the write operation is successful, False otherwise.
        >>> xml_processor = XMLProcessor('test.xml')
        >>> root = xml_processor.read_xml()
        >>> success = xml_processor.write_xml('output.xml')
        >>> print(success)
        True
        """
```

Instruction 3

```
def write_xml(self, file_name):
        """
        Develop a method to store XML content in a given file 'file_name'.
        If the operation succeeds, return 'True', otherwise return 'False'.
        :param file_name: string, the name of the file to write the XML data.
        :return: bool, True if the write operation is successful, False otherwise.
        """
```

14. In a few words, what is the task this function aims to implement? *

_____

15. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

16. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

17. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 5:

Below are instructions for implementing the function:

Instruction 1

```python
from typing import List

def factorize(n: int) -> List[int]:
    """
    Write a function named 'factorize' that takes an integer as an input
    and returns a list of its prime factors sorted from the smallest to
    the largest. This function should include each prime factor according
    to the number of times it divides the input number until it no longer
    can be divided by that prime factor without leaving a remainder.
    The function iterates from 2 up to the square root of the number,
    checking if the current number divides the input number without leaving
    a remainder. If it does, that factor is added to the list, and the input
    number is divided by this factor, repeating the process with the same
    factor until it no longer divides the input number. If the remainder
    after the division is still greater than 1, it is added as the last factor,
    ensuring the product of all listed factors equals the original number.
    """
```

Instruction 2

```python
from typing import List

def factorize(n: int) -> List[int]:
    """
    Return list of prime factors of given integer in the order
    from smallest to largest.
    Each of the factors should be listed number of times corresponding
    to how many times it appeares in factorization.
    Input number should be equal to the product of all factors
    >>> factorize(8)
    [2, 2, 2]
    >>> factorize(25)
    [5, 5]
    >>> factorize(70)
    [2, 5, 7]
    """
```

Instruction 3

```python
from typing import List

def factorize(n: int) -> List[int]:
    """
    Design a 'factorize' function that requires an integer 'n' and produces 'fact',
    a list sorted with prime factors from smallest to largest. Begin from prime 2,
    checking up to the square root of 'n' incremented by one. In each iteration,
    if 'n' is divisible by 'i' ('n % i == 0'), add 'i' to 'fact' and divide 'n'
    by 'i', repeat this for the same 'i' until 'n % i != 0'. Should 'n' still be
    greater than 1 after the iterations, add 'n' to 'fact'. This function returns 'fact'.
    """
```

18. In a few words, what is the task this function aims to implement? *

19. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

20. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

21. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 6:

Below are instructions for implementing the function:

Context

```python
import urllib.parse
class UrlPath:
    """
    The  class is a utility for encapsulating and manipulating the path component of a URL,
    including adding nodes, parsing path strings, and building path strings with optional encoding.
    """

    def __init__(self):
        """
        Initializes the UrlPath object with an empty list of segments
        and a flag indicating the presence of an end tag.
        """
        self.segments = []
        self.with_end_tag = False

    def add(self, segment):
        pass

    @staticmethod
    def fix_path(path):
        pass
```

Instruction 1

```python
def parse(self, path, charset):
        """
        Take the string 'path' and populate 'self.segments' using the decoding
        character set 'charset'. Initially confirm 'path' isn't empty and
        concludes with '/', setting the end tag flag to True. Subsequently,
        streamline 'path' by removing redundant slashes. If the streamlined
        'path' is not void, segment it via '/' to derive distinct elements.
        Employ 'charset' to decode every element from URL encoding.
        :param path: str, the path string to parse.
        :param charset: str, the character encoding of the path string.
        """
```

Instruction 2

```python
def parse(self, path, charset):
        """
        Parses a given path string and populates the list of
        segments in the UrlPath.
        :param path: str, the path string to parse.
        :param charset: str, the character encoding of the path string.
        >>> url_path = UrlPath()
        >>> url_path.parse('/foo/bar/', 'utf-8')

        url_path.segments = ['foo', 'bar']
        """
```

Instruction 3

```
def parse(self, path, charset):
        """
        Parse the given path string ""path"" and populate the list
        ""self.segments"". Use the character encoding ""charset""
        for decoding the segments. Initially, check if ""path""
        is not empty and ends with a '/', setting end tag variable
        to True if it does. Then, modify the ""path"" to remove any
        leading or trailing slashes. If the cleaned ""path"" is not empty,
        split it by '/' to get individual segments. Decode each segment
        from the URL encoding using the specified ""charset"".
        :param path: str, the path string to parse.
        :param charset: str, the character encoding of the path string.
        """
```

22. In a few words, what is the task this function aims to implement? *

_____

23. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ( ) Very Easy
- ( ) Easy
- ( ) Not Easy Nor Difficult
- ( ) Difficult
- ( ) Very Difficult

24. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

25. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 7:

Below are instructions for implementing the function:

Instruction 1

```
def solve(N):
    """
    Create a function named 'solve' which, given a positive integer N,
    converts N into a string for iterating over each digit. These digits
    are then converted into integers to calculate their cumulative sum.
    Afterward, the function should convert this cumulative sum into binary
    format, strip the '0b' prefix found in Python binary literals, and
    deliver the binary string as the output.
    """
```

Instruction 2

```
def solve(N):
    """
    Given a positive integer N, return the total sum of its digits in binary.

    Example
        For N = 1000, the sum of digits will be 1 the output should be "1".
        For N = 150, the sum of digits will be 6 the output should be "110".
        For N = 147, the sum of digits will be 12 the output should be "1100".

    Variables:
        @N integer
             Constraints: 0 ≤ N ≤ 10000.
    Output:
         a string of binary number
    """
```

Instruction 3

```
def solve(N):
    """
    Develop a function called 'solve' that, for a specified positive integer N,
    converts it to a string to access each digit, transforms these digits back
    into integers, and computes their total sum. Following this, the function
    should convert the total sum to its binary form, omitting the '0b' prefix
    typical in Python binary representations, and return the resulting string.
    """
```

26. In a few words, what is the task this function aims to implement? *

27. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

28. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

29. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 8:

Below are instructions for implementing the function:

Context

```python
import re
class RegexUtils:
    """
    The class provides to match, find all occurrences, split, and substitute text using
    regular expressions. It also includes predefined patterns, validating phone numbers
    and extracting email addresses.
    """

    def match(self, pattern, text):
        pass

    def split(self, pattern, text):
        pass

    def sub(self, pattern, replacement, text):
        pass

    def generate_email_pattern(self):
        pass

    def generate_phone_number_pattern(self):
        pass

    def generate_split_sentences_pattern(self):
        pass

    def split_sentences(self, text):
        pass

    def validate_phone_number(self, phone_number):
        pass

    def extract_email(self, text):
        pass


```

Instruction 1

```python
def findall(self, pattern, text):
        """
        Compile a list of strings from all matches found in the given
        'text' that adhere to the specified 'pattern'. Utilizing
        're.findall()' from the 're' module, the function examines
        the 'text' for any part that fits the 'pattern'. These matches
        are gathered into a list that constitutes the function's return value.
        :param pattern: string, Regular expression pattern
        :param text: string, Text to match
        :return: list of string, List of all matching substrings
        """
```

Instruction 2

```
def findall(self, pattern, text):
      """
      Find all matching substrings and return a list of all matching substrings
      :param pattern: string, Regular expression pattern
      :param text: string, Text to match
      :return: list of string, List of all matching substrings
      >>> ru = RegexUtils()
      >>> ru.findall(r'\b\d{3}-\d{3}-\d{4}\b',
            "123-456-7890 abiguygusu 876-286-9876 kjgufwycs 987-762-9767")
      ['123-456-7890', '876-286-9876', '987-762-9767']
      """
```

Instruction 3

```
def findall(self, pattern, text):
      """
      Return a list of strings containing every match found
      in the 'text' for the given 'pattern'.
      :param pattern: string, Regular expression pattern
      :param text: string, Text to match
      :return: list of string, List of all matching substrings
      """
```

30. In a few words, what is the task this function aims to implement? *

31. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

32. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

33. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 9:

Below are instructions for implementing the function:

Instruction 1

```
def int_to_mini_roman(number):
    """
    Given a positive integer, obtain its roman
    numeral equivalent as a string,
    and return it in lowercase.
    Restrictions: 1 <= num <= 1000

    Examples:
    >>> int_to_mini_roman(19) == 'xix'
    >>> int_to_mini_roman(152) == 'clii'
    >>> int_to_mini_roman(426) == 'cdxxvi'
    """
```

Instruction 2

```
def int_to_mini_roman(number):
    """
    Create a function labelled 'int_to_mini_roman' that turns a
    specified positive integer into its Roman numeral equivalent,
    delivered in lowercase. This function supports numbers from
    1 through 1000.
    """
```

Instruction 3

```
def int_to_mini_roman(number):
    """
    Construct a function called 'int_to_mini_roman' which translates
    a given positive integer into a Roman numeral, returned as a
    lowercase string, and can handle integers from 1 up to 1000.
    """
```

34. In a few words, what is the task this function aims to implement? *

35. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

36. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

37. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 10:

Below are instructions for implementing the function:

Context

```python
from datetime import datetime
class Chat:
    """
    This is a chat class with the functions of adding users, removing users,
    sending messages, and obtaining messages.
    """

    def __init__(self):
        """
        Initialize the Chat with an attribute users, which is an empty dictionary.
        """
        self.users = {}

    def remove_user(self, username):
        pass

    def send_message(self, sender, receiver, message):
        pass

    def get_messages(self, username):
        pass



```

Instruction 1

```python
def add_user(self, username):
        """
        For adding a new user to a Chat, scrutinize whether the 'username'
        is listed in 'self.users'. Return 'False' if it is, implying duplication.
        Otherwise, record the 'username' in 'self.users' by initializing its value
        to an empty list, indicative of an empty message log, before returning 'True'.
        This setup permits effective user integration and preparation for upcoming
        communications.
        :param username: The user's name, str.
        :return: If the user is already in the Chat, returns False, otherwise, returns True.
        """
```

Instruction 2

```
def add_user(self, username):
        """
        Add a new user to the Chat.
        :param username: The user's name, str.
        :return: If the user is already in the Chat, returns False,
        otherwise, returns True.
        >>> chat = Chat()
        >>> chat.add_user('John')
        True
        self.users = {'John': []}
        >>> chat.add_user('John')
        False
        """
```

Instruction 3

```
def add_user(self, username):
        """
        Insert a new user into the Chat by validating the 'username'.
        Return 'False' if the 'username' already exists; otherwise,
        record the 'username' and return 'True'.
        :param username: The user's name, str.
        :return: If the user is already in the Chat, returns False,
                otherwise, returns True.
        """
```

38. In a few words, what is the task this function aims to implement? *

_____

39. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

40. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

41. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 11:

Below are instructions for implementing the function

Instruction 1

```
def find_max(words):
    """
    Write a function that accepts a list of strings.
    The list contains different words. Return the word with maximum number
    of unique characters. If multiple strings have maximum number of unique
    characters, return the one which comes first in lexicographical order.

    find_max(["name", "of", "string"]) == "string"
    find_max(["name", "enam", "game"]) == "enam"
    find_max(["aaaaaaa", "bb" ,"cc"]) == ""aaaaaaa"
    """
```

Instruction 2

```
def find_max(words):
    """
    Implement a function titled 'find_max' that accepts an array of strings.
    It must identify and return the word with the maximum distinct characters.
    Should a tie occur among multiple words, it will choose the one
    lexicographically first. To find this, the function iterates over the words,
    counts the unique characters for each, and updates the highest count encountered
    so far if needed.
    """
```

Instruction 3

```
def find_max(words):
    """
    Create a function called 'find_max' that accepts a list of strings.
    This function should identify and return the string with the most
    distinct characters. If there are several strings with equivalent
    counts of unique characters, the function must return the earliest
    one in alphabetical order. This is accomplished by traversing each
    string, counting its unique characters, and maintaining record of
    the highest count encountered.
    """
```

42. In a few words, what is the task this function aims to implement? *

43. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- [ ] Very Easy
- [ ] Easy
- [ ] Not Easy Nor Difficult
- [ ] Difficult
- [ ] Very Difficult

44. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- [ ] 1 min
- [ ] 5 min
- [ ] 10 min
- [ ] 20 min
- [ ] 40 min
- [ ] > 40 min

45. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 12:

Below are instructions for implementing the function:

Context

```python
class EightPuzzle:
    """
    This class is an implementation of the classic 8-puzzle game,
    including methods for finding the blank tile, making moves,
    getting possible moves, and solving the puzzle using a breadth-first search algorithm.
    """

    def __init__(self, initial_state):
        """
        Initializing the initial state of Eight Puzzle Game,
        stores in attribute self.initial_state.
        And set the goal state of this game, stores in self.goal_state.
        In this case, set the size as 3*3
        :param initial_state: a 3*3 size list of Integer, stores the initial state
        """
        self.initial_state = initial_state
        self.goal_state = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]

    def find_blank(self, state):
        pass

    def move(self, state, direction):
        pass

    def solve(self):
        pass
```

Instruction 1

```python
def get_possible_moves(self, state):
        """
        Calculate which movements ('up', 'down', 'left', 'right')
        are permissible for the blank tile from its present position in the puzzle.
        The functionality starts by pinpointing where this blank tile is positioned.
        Depending on the tile's specific row and column, it then assesses the
        applicable movements, verifying that it's not on the first row for the 'up'
        move and confirming suitable conditions are met for 'down', 'left', and 'right'.
        :param state: a 3*3 size list of Integer, stores the current state.
        :return moves: a list of str, store all the possible moving directions according
        to the current state.
        """
```

Instruction 2

```
def get_possible_moves(self, state):
        """
        According the current state, find all the possible moving directions.
        Only has 4 direction 'up', 'down', 'left', 'right'.
        :param state: a 3*3 size list of Integer, stores the current state.
        :return moves: a list of str, store all the possible moving
                                directions according to the current state.
        >>> eightPuzzle.get_possible_moves([[2, 3, 4], [5, 8, 1], [6, 0, 7]])
        ['up', 'left', 'right']
        """
```

Instruction 3

```
def get_possible_moves(self, state):
        """
        According to the current state, identify all possible movement
        directions for the blank tile which can be 'up', 'down', 'left', 'right'.
        :param state: a 3*3 size list of Integer, stores the current state.
        :return moves: a list of str, store all the possible moving directions
                                                according to the current state.
        """
```

46. In a few words, what is the task this function aims to implement? *

47. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ( ) Very Easy
- ( ) Easy
- ( ) Not Easy Nor Difficult
- ( ) Difficult
- ( ) Very Difficult

48. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- [ ] 1 min
- [ ] 5 min
- [ ] 10 min
- [ ] 20 min
- [ ] 40 min
- [ ] > 40 min

49. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Demographics Information

50. What is your highest education qualification? *

Une seule réponse possible.

- [ ] Ph.D.
- [ ] Master's Degree
- [ ] Bachelor Degree
- [ ] Autre : _____

51. What is your field of work? (Engineer, Researcher, Developer, etc.) *

_____

52. How many years of development experience do you have? *

Une seule réponse possible.

- [ ] Less than 5
- [ ] Between 5 and 10
- [ ] More than 10

53. Thank you for completing the survey! Any feedback you would like to provide?

_____

_____

_____

_____

_____